

Unit 4.1: Foundations of Predictive Modeling

DAIR-3 Workshop, Summer 2026

Presented by Suraj Rampure (rampur@umich.edu)

Introduction 🙌

- My name is Suraj Rampure.

My first name is pronounced "sooh-rudge".

- I am Teaching Faculty in Computer Science and Engineering at the University of Michigan.
- Recently, I've taught *Mathematics for Machine Learning* and *Practical Data Science*.

Infrastructure

- All of the content I will present can be found in
https://github.com/DAIR3/DAIR3-Workshop/tree/main/resources/unit_4
- Each "block" (4 total) has one Jupyter Notebook, which contains both the slides I will present and activities for you to work on.
- **Have these Jupyter Notebooks open in VSCode while I'm presenting!**
If you have trouble accessing or running them, let me know.
- At the end of each block, you will write (1) write code and (2) submit reflections to ALICE.
<https://utsa-alice.org>

Python environment

- The Unit 4 notebooks use these code packages:

```
numpy  
pandas  
plotly  
nbformat  
scikit-learn  
seaborn
```

- Install `scikit-learn`, but import it as `sklearn`. `nbformat` is included because Plotly uses it to render figures inside notebooks.
- If you can already open and run notebooks in VS Code/Jupyter, this list should be sufficient for the Unit 4 code.

Before: Inference

- In Unit 3: **Rigorous Statistical Design**, you were developing scientific research aims and analytic plans.
- The emphasis was on **statistical inference**.
You studied estimation, effect sizes, uncertainty, and causal/mechanistic reasoning.
- You were explicitly **not** trying to make "high prediction accuracy" the goal.

Now: Prediction

- In Unit 4: **Designing Interpretable Predictive Models**, prediction **is** the goal.

The question shifts from "what can we infer about the relationships between variables?" to "what can we predict about new observations, using only information available at prediction time?"

- In our setting, models need to be interpretable and are subject to scrutiny by governing bodies.

We don't *just* care about maximizing performance numbers, which is why we can't just use LLMs and neural networks for everything.

- We will start by introducing the foundations of predictive modeling, then focus on
 - Ensuring our models generalize well to unseen data.
 - Selecting features in an interpretable and reproducible manner.
 - Documenting our decisions.

What is machine learning?

Machine learning is about automatically learning patterns from data.

For example, a handwritten digit recognizer can be trained on thousands of labeled images of digits. Each label tells the model the correct digit for one image; training is the process of learning patterns that let the model make predictions on new images.

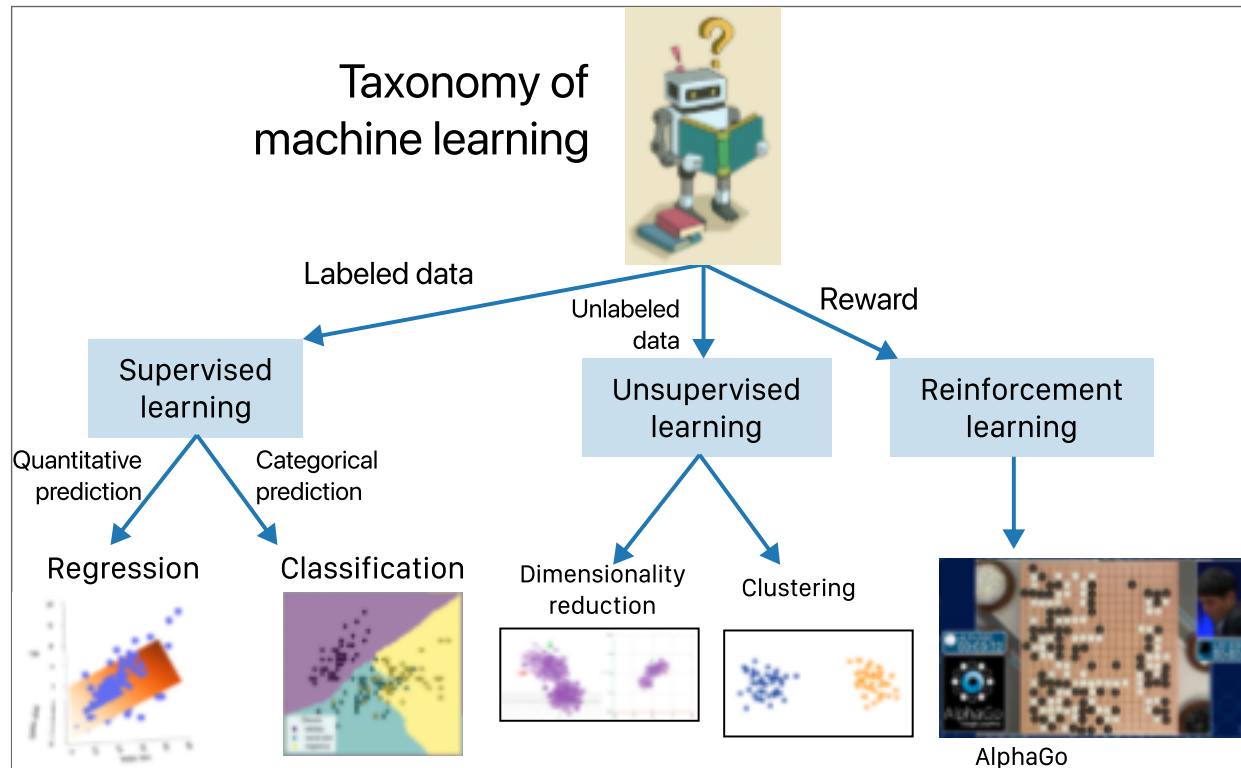


A few of the images in the MNIST dataset.

The MNIST database contains 70,000 grayscale images of handwritten digits. Each image is 28 by 28 pixels and has a true label from 0 to 9.

Taxonomy of machine learning

- Unit 4 lives mostly in **supervised learning**: we have examples with inputs and a target.
- Specifically, we'll focus on **regression**, where the aim is to predict a real-valued target.



Regression in `sklearn`

Loading the data

- Run the cell below to load in a toy dataset that we'll start with.
- Our features will be `departure_hour` and `day_of_month`; our target variable is commute time in `minutes`.

"Feature" is another term for "covariate".

In [2]:

```
df = pd.read_csv('data/commute-times.csv')
df['day_of_month'] = pd.to_datetime(df['date']).dt.day
df[['departure_hour', 'day_of_month', 'minutes']]
```

Out[2]:

	departure_hour	day_of_month	minutes
0	10.816667	15	68.0
1	7.750000	16	94.0
2	8.450000	22	63.0
3	7.133333	23	100.0
4	9.150000	30	69.0
...	...	...	...
60	7.516667	27	85.0
61	7.550000	29	91.0
62	7.583333	4	68.0
63	7.450000	5	90.0
64	7.600000	7	83.0

65 rows × 3 columns

Identifying the prediction problem

- **Goal:** predict commute time in minutes .
- **Why?** To make commute time predictions for future days, perhaps to inform when we might leave for work.
- **Unit of analysis:** one day.
- **Data collection:** logs from one individual across several years.

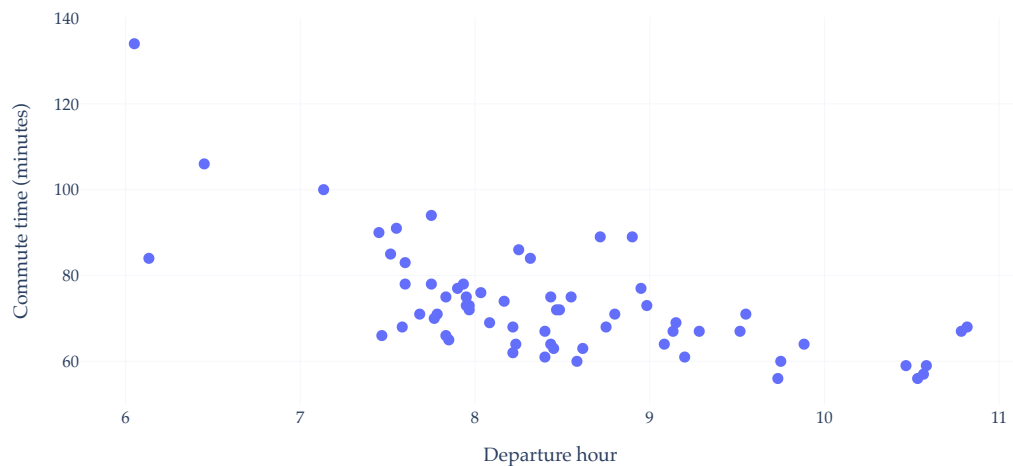
Visualizing commute time

- Before fitting a model, we should visualize relationships between the predictor and target.
- Here, each point is one commute, with departure hour on the x -axis and commute time on the y -axis.

In [3]:

```
px.scatter(  
    df,  
    x='departure_hour',  
    y='minutes',  
    title='Commute time vs. departure hour',  
    labels={  
        'departure_hour': 'Departure hour',  
        'minutes': 'Commute time (minutes)',  
    },  
    hover_data=['date', 'day'],  
)
```

Commute time vs. departure hour



sklearn

- `sklearn` (scikit-learn) is an industry-standard Python library for non-deep machine learning.



- It interfaces with `numpy` arrays, and to an extent, `pandas` DataFrames.
- Huge benefit: the **documentation online** is excellent.

The `LinearRegression` class

- `sklearn` comes with several subpackages, including `linear_model` and `tree`, each of which contains several classes of models.
- We'll start with the `LinearRegression` class from `linear_model`.
- **Important:** From [the documentation](#), we have:

LinearRegression fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

- It is important to understand the (hidden!) assumptions being made when fitting a model.

In [4]:

```
from sklearn.linear_model import LinearRegression
```

Fitting a multiple linear regression model

- Let's use `sklearn` to find the optimal **parameters** for the following model:

$$\begin{aligned}\text{pred. commute}_i &= h(\text{departure hour}_i, \text{day of month}_i) \\ &= w_0 + w_1 \cdot \text{departure hour}_i + w_2 \cdot \text{day of month}_i\end{aligned}$$

- First, we must instantiate a `LinearRegression` object and **train it**.

In [5]:

```
model_multiple = LinearRegression()  
model_multiple
```

Out[5]:

▼ LinearRegression ⓘ ?
LinearRegression()

In [6]:

```
# This line trains our model.  
model_multiple.fit(X=df[['departure_hour', 'day_of_month']], y=df['minutes'])
```

Out[6]:

▼ LinearRegression ⓘ ?
LinearRegression()

Accessing fit coefficients

- After fitting, we can access the best intercept (w_0^*) and coefficients (w_1^*, w_2^*).
- These coefficients tell us that the "best way" (according to squared loss) to make commute time predictions using a linear model is using:

$$\text{pred. commute}_i = 141.86 - 8.22 \cdot \text{departure hour}_i + 0.06 \cdot \text{day of month}_i$$

In [7]:

```
model_multiple.intercept_, model_multiple.coef_
```

Out[7]:

```
(141.86402699471932, array([-8.2233821 ,  0.05615985]))
```

- **This is not a causal statement!**
- Does this say that departure time is a more useful predictor than day of the month? Not necessarily: it depends on the scale of the data.

What if I measured departure time in minutes from 12:00 instead of hours?

Discuss: Where did these optimal parameters come from?

- How did `sklearn` find these values? How would you explain where they came from to someone not familiar with machine learning?

In [8]:

```
model_multiple.intercept_, model_multiple.coef_
```

Out[8]:

```
(141.86402699471932, array([-8.2233821 ,  0.05615985]))
```

Making predictions

- Fit `LinearRegression` objects have a `predict` method, which can be used to predict commute times given a `departure_hour` and `day_of_month`.
- In modern ML/AI, this step is (confusingly) called "inference".

In [9]:

```
# What if I leave at 9:15AM on the 26th of the month?
model_multiple.predict(pd.DataFrame({
    'departure_hour': [9.25],
    'day_of_month': [26],
}))
```

Out[9]:

```
array([67.25789852])
```

In [10]:

```
model_multiple.predict(pd.DataFrame({'departure_hour': [9.25], 'day_of_month': [26]}))
```

Out[10]:

```
array([67.25789852])
```

In [11]:

```
departure_grid = np.linspace(df['departure_hour'].min(), df['departure_hour'].max(), 25)
day_grid = np.linspace(df['day_of_month'].min(), df['day_of_month'].max(), 25)
departure_mesh, day_mesh = np.meshgrid(departure_grid, day_grid)

prediction_grid = pd.DataFrame({
    'departure_hour': departure_mesh.ravel(),
    'day_of_month': day_mesh.ravel(),
})
minute_mesh = model_multiple.predict(prediction_grid).reshape(departure_mesh.shape)

fig = go.Figure()
fig.add_trace(go.Scatter3d(
    x=df['departure_hour'],
    y=df['day_of_month'],
    z=df['minutes'],
    mode='markers',
    name='Observed commutes',
    marker={'size': 5, 'opacity': 0.8},
))
fig.add_trace(go.Surface(
    x=departure_mesh,
    y=day_mesh,
    z=minute_mesh,
    name='Plane of best fit',
    colorscale='Oranges',
    opacity=0.55,
    showscale=False,
))
fig.update_layout(
    title='Plane of best fit',
    scene={
        'xaxis_title': 'Departure hour',
        'yaxis_title': 'Day of month',
        'zaxis_title': 'Commute time (minutes)',
    },
    margin={'l': 0, 'r': 0, 'b': 0, 't': 50},
)
fig
```

Plane of best fit

LinearRegression class summary

Property	Example	Description
Initialize model parameters	<code>lr = LinearRegression()</code>	Create an empty linear regression model.
Fit the model to the data	<code>lr.fit(X, y)</code>	Determine regression coefficients from training data.
Use model for prediction	<code>lr.predict(X_new)</code>	Use the fitted regression equation to make predictions.
Access fitted attributes	<code>lr.coef_, lr.intercept_</code>	Inspect the fitted coefficients and intercept.

Evaluating model performance

- Since we're going to start to fit lots of different models to the commute times dataset, it'll be useful to compare their **mean squared errors**.

$$\text{mean squared error} = \frac{1}{n} \sum_{i=1}^n (y_i - h(\vec{x}_i))^2$$

where y_i is the i -th target value and $h(\vec{x}_i)$ is the i -th predicted target value.

- `sklearn` has a built-in `mean_squared_error` function.

Remember, the units of MSE are the units of y , squared. So the value below is 96.78 **minutes²**.

In [12]:

```
((df['minutes'] - model_multiple.predict(df[['departure_hour', 'day_of_month']))) ** 2).mean()
```

Out[12]:

```
96.78730488437492
```

In [13]:

```
# Same result.  
from sklearn.metrics import mean_squared_error  
mean_squared_error(df['minutes'], model_multiple.predict(df[['departure_hour', 'day_of_month'])))
```

Out[13]:

```
96.78730488437492
```

Discuss: What is the single best constant prediction?

We're soon going to create models of increasing sophistication. To compare, it's useful to have a baseline.

1. What **constant value** w^* minimizes mean squared error,

$$R(w) = \frac{1}{n} \sum_{i=1}^n (y_i - w)^2$$

2. What is the mean squared error of the model that uses this w^* ?
3. Would it make sense to use this w^* for predicting commute times?

Comparing models

- Let's construct a dictionary to keep track of the MSEs we've seen so far.
- To compare, let's also fit and measure a simple linear model (only using `departure_hour`) and constant model (same prediction for all observations).

In [14]:

```
mse_dict = {}
mse_dict['departure_hour + day_of_month'] = mean_squared_error(
    df['minutes'], model_multiple.predict(df[['departure_hour', 'day_of_month']])
)

# Simple linear model.
model_simple = LinearRegression()
model_simple.fit(X=df[['departure_hour']], y=df['minutes'])
mse_dict['departure_hour'] = mean_squared_error(df['minutes'], model_simple.predict(df[['departure_hour']]))

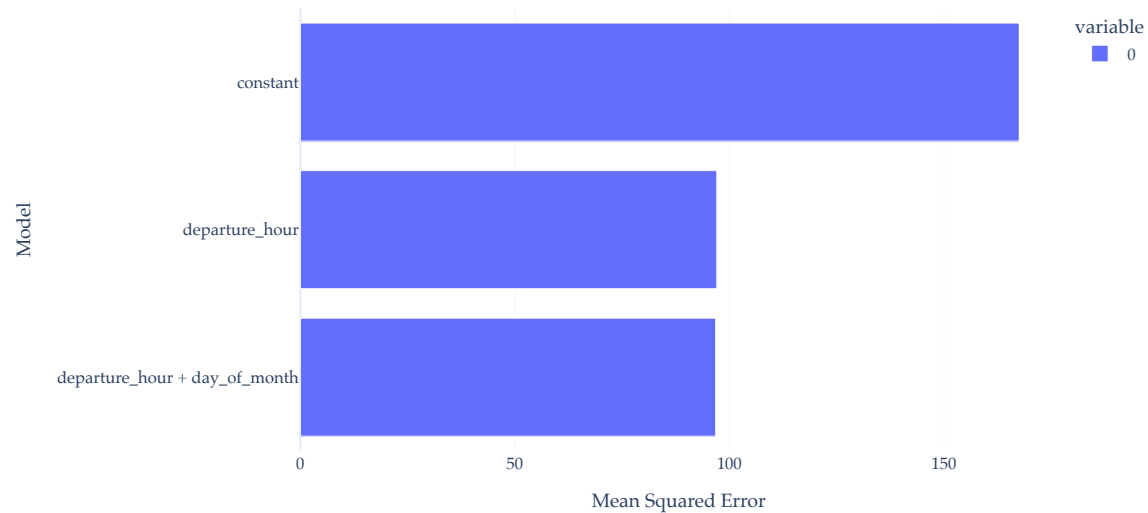
# Constant model.
model_constant = df['minutes'].mean() # Best constant when using mean squared error is the mean.
mse_dict['constant'] = mean_squared_error(df['minutes'], np.ones(df.shape[0]) * model_constant)
```


Discuss: Is adding more features always a good thing?

- As we can see, adding `day_of_month` as a feature **barely** reduced our model's MSE.

In [15]:

```
pd.Series(mse_dict).plot(kind='barh').update_layout(xaxis_title='Mean Squared Error', yaxis_title='Model')
```



- When might it be a bad idea to add more features?

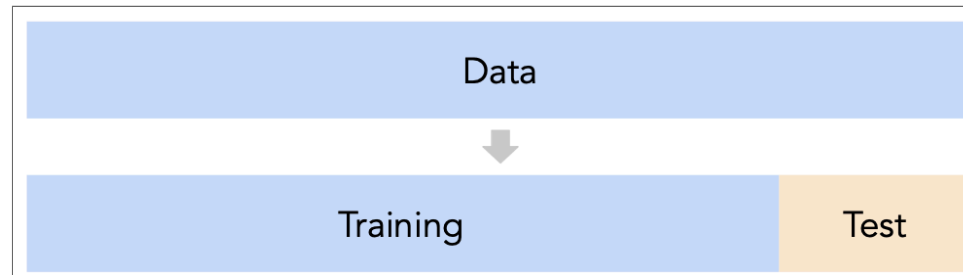
Generalization

What's the point?

- The goal of building a predictive model is to make reliable predictions **on unseen data**.
- In order to ensure our models **generalize**, we can't just look at its performance on the data we used to fit it.

Train-test splits 🚂

- Suppose we're choosing between many different models.
- We won't know whether a model has **overfit** to our sample unless we get to see how well it performs on a new sample from the same population.
- 💡 **Idea**: split our dataset into a **training set** and **test set**.



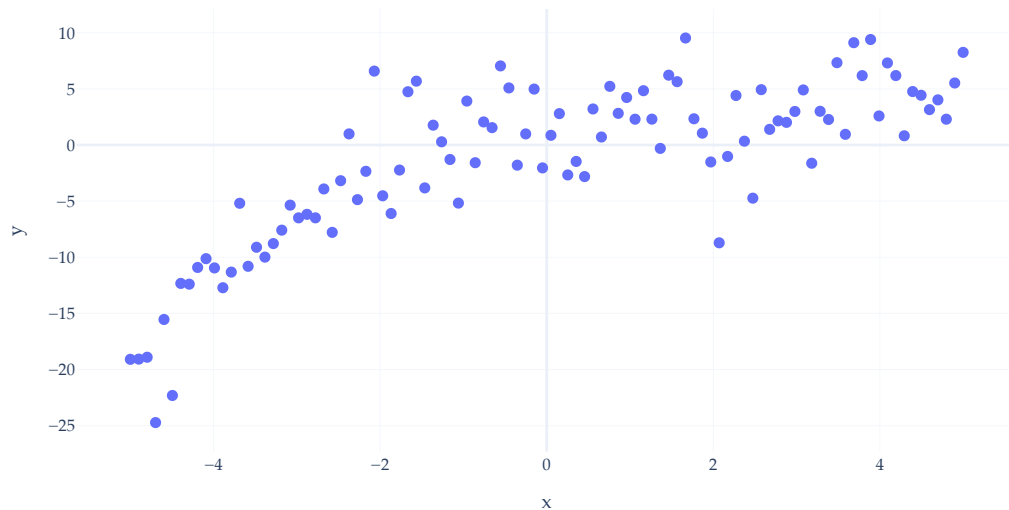
- For each model we're considering:
 - Use **only** the training set to fit that model (i.e. find w^* 's).
 - Use the test set to evaluate that model's error (e.g. compute its MSE).
- Pick the model with the **best** test set performance.
- Why? The test set is like a new sample of data from the same population as the training data.

Example: Polynomial regression

- To elaborate on this point, we'll use a different synthetic dataset.
- The data were generated from a nonlinear relationship with noise.

In [16]:

```
sample_1 = make_polynomial_sample(n=100)
fig = px.scatter(x=sample_1['x'], y=sample_1['y'])
fig
```



Polynomial fits on sample

- Before introducing a train/test split, let's treat the entire sample as training data (a sin in practice).
- Drag the slider to see how the fitted polynomial changes as the degree ranges from 1 to 30.
- A more flexible model can fit the training data more closely, but that does not automatically mean it will predict new data well.

In [17]:

```
degree_grid = np.arange(1, 31)
xs = pd.DataFrame({'x': np.linspace(sample_1['x'].min(), sample_1['x'].max(), 300)})

fig = px.scatter(sample_1, x='x', y='y', title='Polynomial fit: use the slider to change model flexibility')

for degree in degree_grid:
    model = make_pipeline(StandardScaler(), PolynomialFeatures(degree, include_bias=False), LinearRegression())
    model.fit(sample_1[['x']], sample_1['y'])
    fitted = model.predict(xs)
    fig.add_scatter(
        x=xs['x'],
        y=fitted,
        mode='lines',
        name=f'Degree {degree}',
        line={'color': 'orange', 'width': 4},
        visible=(degree == 1),
    )

steps = []
for i, degree in enumerate(degree_grid):
    visible = [True] + [False] * len(degree_grid)
    visible[i + 1] = True
    steps.append({
        'method': 'update',
        'label': str(degree),
        'args': [
            {'visible': visible},
            {'title': f'Polynomial fit: degree {degree}'},
        ],
    })

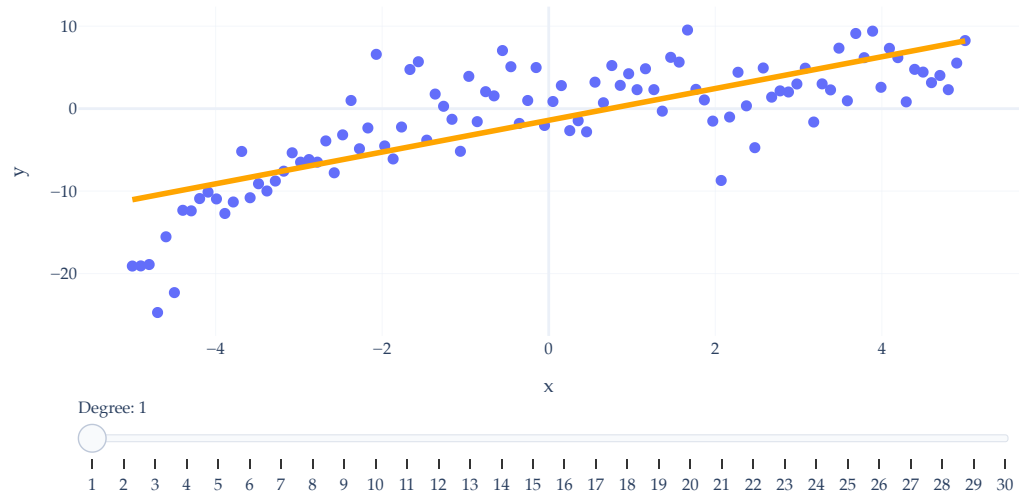
fig.update_layout(
    showlegend=False,
    sliders=[{
        'active': 0,
```

```

    'currentvalue': {'prefix': 'Degree: '},
    'pad': {'t': 40},
    'steps': steps,
  }],
)
fig

```

Polynomial fit: use the slider to change model flexibility



Implementing a train-test split

In [18]:

```
from sklearn.model_selection import train_test_split
```

In [19]:

```
# train_test_split?
```

In [20]:

```
X = sample_1[['x']]
y = sample_1[['y']]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=23)
```

In [21]:

```
print('Rows in X_train:', X_train.shape[0])
display(X_train.head())
print('Rows in X_test:', X_test.shape[0])
display(X_test.head())
```

Rows in X_train: 80

	x
85	3.585859
28	-2.171717
8	-4.191919
11	-3.888889
63	1.363636

Rows in X_test: 20

x	
26	-2.373737
80	3.080808
82	3.282828
68	1.868687
77	2.777778

Training on *only* the training data

- Ignore most of the code below; we'll look more closely at it soon.

In [22]:

```
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
train_errs = []
test_errs = []
for d in range(1, 31):
    pl = make_pipeline(StandardScaler(), PolynomialFeatures(d, include_bias=False), LinearRegression())
    pl.fit(X_train, y_train)
    train_errs.append(mean_squared_error(y_train, pl.predict(X_train)))
    test_errs.append(mean_squared_error(y_test, pl.predict(X_test)))
errs = pd.DataFrame({'Train Error': train_errs, 'Test Error': test_errs})
errs
```

Out[22]:

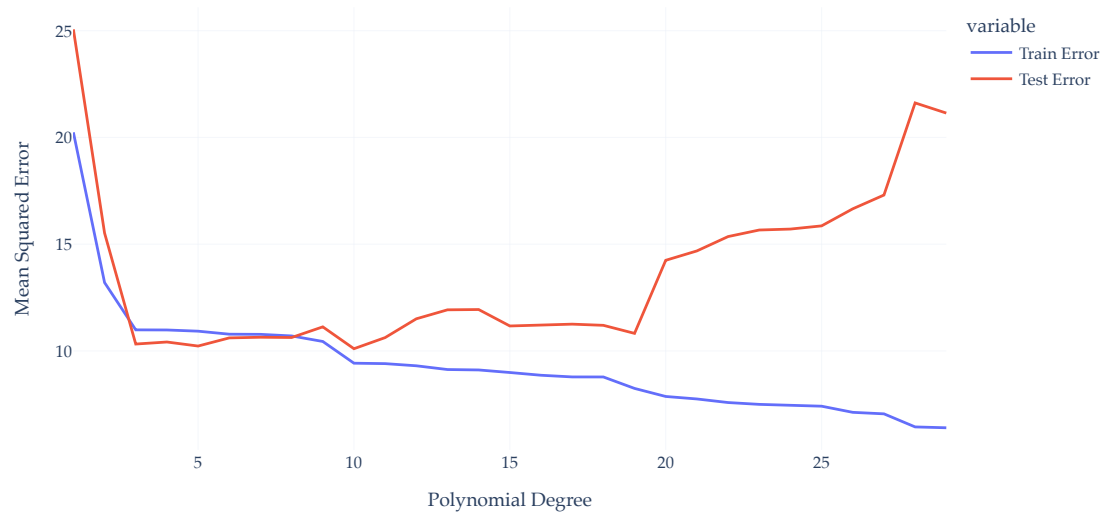
	Train Error	Test Error
0	20.240161	25.063581
1	13.202878	15.522270
2	10.987288	10.323537
3	10.982120	10.418200
4	10.926117	10.229075
5	10.784634	10.610924
6	10.777538	10.641458
7	10.702730	10.628845
8	10.444038	11.127926
9	9.425691	10.104174
10	9.405468	10.625440
11	9.304490	11.505535
12	9.128978	11.926229
13	9.109209	11.939462
14	8.989578	11.169803
15	8.861438	11.217916
16	8.783115	11.255141
17	8.781113	11.200002
18	8.244470	10.823129
19	7.867459	14.245390
20	7.750624	14.684867
21	7.582804	15.357044
22	7.497555	15.665027
23	7.474194	15.709195
24	7.411505	15.858603

	Train Error	Test Error
25	7.123864	16.657791
26	7.053291	17.302198
27	6.441010	21.618531
28	6.400843	21.142720
29	6.391451	20.704237

Discuss: Which degree should we choose?

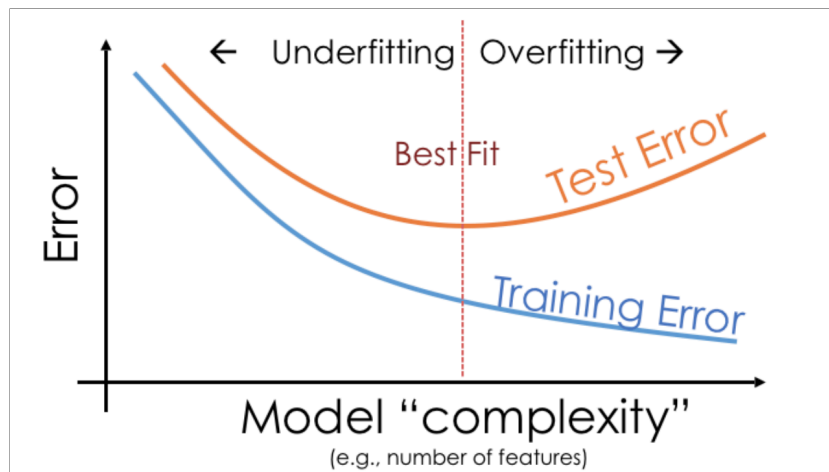
In [23]:

```
errs.index = errs.index + 1  
fig = px.line(errs.iloc[: -1])  
fig.update_layout(showlegend=True, xaxis_title='Polynomial Degree', yaxis_title='Mean Squared Error')  
fig
```



Training error vs. test error

- The pattern we saw in the previous example is true more generally.



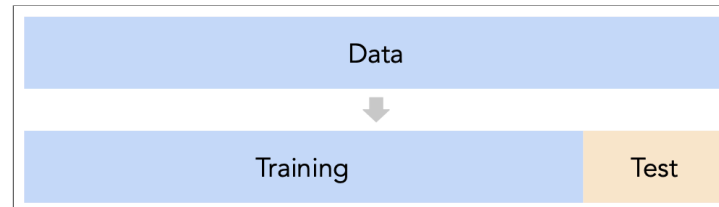
- We pick the models at the "valley" of test error.
- Note that training error **tends** to underestimate test error, but it doesn't have to – i.e., it is possible for test error to be lower than training error (say, if the test set is "easier" to predict than the training set).
- These patterns hold true for "classic" machine learning models, like the ones we're studying here. But in deep neural networks, this pattern is often violated; extremely complex models can have low test error as well. This phenomenon is known as "double descent"; learn more [here](#).

How is this relevant to biomedical examples?

- It *may* be the case that you need to tune a **hyperparameter**, like polynomial degree. The theory we just discussed applies there.
- Instead, it may be the case that you have several candidate sets of features that you'd like to choose between.

Conducting train-test splits

- Recall, **training data** is used to fit our model, and **test data** is used to evaluate our model.



- Random split:** randomly assign rows to train/test. This is the default `train_test_split` behavior.
- Time split:** train on earlier observations and test on later observations. Use this when the goal is **forecasting future data**.
- Group split:** keep entire groups together, e.g. train on some counties and test on held-out counties. Use this when rows within a group are correlated.
- Stratified split:** preserve the distribution of an important categorical variable or rare outcome across train/test sets.
- The split should match the way the model will actually be used.

Data leakage

- **Data leakage** occurs when information that would not be available at prediction time is used while training, tuning, or evaluating a model.
- Leakage can make held-out performance look better than it really is, even if we used a train/test split.
- Common biomedical examples:
 - preprocessing the full dataset before splitting, so test-set summaries influence the training data;
 - letting records from the same patient, hospital, county, or time period appear in both train and test;
 - using variables recorded after the outcome, or variables that are downstream consequences of the outcome;
 - selecting features or model settings after repeatedly checking test performance.
- Rule of thumb: every choice that learns from data should be made using training data only, then applied unchanged to the test set.

But wait...

- With our current strategy, we are choosing the model that creates the model that **performs best on the test set**.
- As such, we are **overfitting to the test set** – the best model on the test set might not be the best model on a totally unseen dataset.
- It seems like we need **another** split.
- The real solution is called **cross-validation**. We'll discuss it later in Unit 4.

Example: iBudget Data

Before you apply what you've learned to the birthweight dataset, let's look at an important case study.

Example: Florida's iBudget program

- iBudget is Florida's Home and Community-Based Services Medicaid waiver for people with developmental disabilities.

The iBudget system, established in 2010, was designed to provide individuals with greater flexibility and

control over their services by allocating funding based on a standardized algorithm. The intent of the iBudget system is to ensure the equitable distribution of resources among individuals with similar needs.

- Read the "Algorithm Report" here:
<https://apd.myflorida.com/resources/reports/APD%20Algorithm%20Report.pdf>
- Keep this open in a tab; we'll refer to it throughout Unit 4.

Discuss: What's the need for a new model?

Answer the following questions:

- What is the target variable?
- On what data was the "current model" trained?
- How does the "current model" perform on recent data?
- How did they measure model performance?

Hint: scroll to Page 9, and scroll from there.

Back to the birthweight dataset 🧒

Reintroducing the birthweight data

- Now that we have seen the `sklearn` syntax and train/test split ideas on toy examples, we can return to the real dataset from earlier sessions.
- Unit 3 used quantitative `birthweight`, measured in grams, as the outcome.
- We will use the same target variable here, but now the goal is predictive performance on held-out births.
- The next cell uses the same prepared `1971.csv.gz` file expected by the Unit 3 notebook.

In [24]:

```
try:
    births = load_birthweight_1971()
    print(f"Loaded {births.shape[0]:,} births and {births.shape[1]:,} columns.")
    display(births.sample(5))
except FileNotFoundError as err:
    births = None
    print(err)
```

Loaded 1,781,774 births and 14 columns.

	year	state	county	smsa	sex	dadrace	momrace	momage	birthorder	dadage	birthweight	plurality	interval	popsiz
1220503	1971	35	009	066	male	White	White	26	2.0	26.0	3289.0	1	36.0	9
891876	1971	26	095	174	male	White	White	28	3.0	30.0	3119.0	1	18.0	9
1204339	1971	34	078	000	male	NaN	Black	24	3.0	NaN	2410.0	1	16.0	9
742666	1971	22	012	031	female	White	White	20	1.0	20.0	2948.0	1	NaN	9
1121043	1971	33	029	137	male	White	White	22	NaN	30.0	3374.0	1	NaN	0

Identifying the prediction problem

- **Goal:** predict `birthweight` in grams.
- **Why?** To make birthweight predictions for future births, which may be used to estimate the number of underweight births per county.
- **Unit of analysis:** a single birth.
- **Data collection:** NCHS collects data on all births in the US in each year.
- Note: predicting `birthweight` itself is a regression problem; predicting whether `birthweight` is below some threshold is a classification problem, and would involve a different sort of model.

Activity: Fit a baseline model

Using the `births` DataFrame below,

1. Create `X` using the features `momage`, `dadage`, `plurality`, and `birthorder`, and create `y` using the target `birthweight`.
2. Perform a train/test split with test size 20%.
3. Fit a multiple linear regression model on the training set using `LinearRegression`.
4. Compute the training and test set MSEs.
5. On ALICE, include the following:
 - A. A screenshot of your code and both MSEs.
 - B. Your thoughts on whether the current model is generalizing well to unseen data.
 - C. Your thoughts on whether a random train/test split makes sense for the current prediction task, and thoughts on how data leakage may be present.

Discuss: what are the units of mean squared error?

In [25]:

```
# Your code goes here...
```